

1 · System Overview — Components & Pipelines

CLIENT TIER — FRONTEND/ (REACT 18 + VITE, SERVED ON :5173)

Candidate Test App :5173

- `useProctoring.ts` — webcam capture via `getUserMedia` → `canvas` → `base64 JPEG`
- `proctorService.ts` — `captureFrame()` (max 640px, q=0.7), `submitAnalysis()`
- `useLiveProctoringPublisher.ts` — publishes cam/mic/screen to LiveKit
- Browser-native signals: `tab-switch`, `fullscreen-exit`, `monitor count`

Admin Dashboard :5173

- `ProctorDashboard.tsx`, `LiveProctoring.tsx`, `TestAIProctoring.tsx`
- Subscribes to `Socket.IO` room `proctor-{testId}` for live violation feed
- Subscribes to `LiveKit` room (`watch-only` token) for live A/V
- Reviews / dismisses events, plays back stored recordings

↓
POST `/api/proctoring/session/:id/analysis` (~every 2.5-4s)
+ POST `.../violation` (discrete browser events)

APPLICATION TIER — BACKEND/ (NODE.JS + EXPRESS + TYPESCRIPT, :3000)

Node/Express Backend :3000 HTTP + Socket.IO server (index.ts)

- `routes/proctoring.ts` → `controllers/proctoring.ts` — `submitAnalysis`, `reportViolation`, `ingestExternalEngineEvent`, admin read endpoints
- `services/proctorAIService.ts` — local heuristics (`face/gaze/audio-disabled/phone-flag`), `trust-score` calculation, event severity/storage
- `services/pythonVisionService.ts` — HTTP bridge to Python CV service `/analyze` (timeout, retry=0, circuit breaker)
- `services/liveProctoringService.ts` — mints `LiveKit` JWTs (`publish` for candidate, `subscribe` for admin)
- `services/fileStorageService.ts` — persists snapshots/recordings to disk + meta JSON
- `Socket.IO` events: `candidate-proctor-join`, `admin-proctor-join`, `violation-detected`, `candidate-status`, `candidate-live-frame` → `live-frame`

External AI Engine optional

- 3rd-party detector, out-of-process
- Pushes events directly via `webhook`
- Auth: `x-ai-key` / `x-api-key` header = `PROCTOR_ENGINE_API_KEY`
- Bypasses the standard analysis loop entirely

POST `/api/proctoring/engine/event` →

↓
POST `/analyze { frame(b64), sessionId, capturedAt }`

↓
Prisma ORM (SQL)

↓
fs read/write

↓
mint JWT (control-plane only)

SERVICE / DATA TIER

Python CV Service :8010

FastAPI · `app.py` (v2.0.0)

- `OpenCV` — `decode/resize/brightness`
- `MediaPipe` `FaceDetection` + `FaceMesh` — `face count`, `gaze`
- `YOLOv8n` (ultralytics) — `phone detection` (`COCO cls 67`)
- Returns violations / face / objects / stats JSON

PostgreSQL via Prisma

- `ProctorSession`, `ProctorEvent`
- `ProctorRecording`, `ViolationClip`
- `FaceSnapshot`, `ProctorTelemetry`

Filesystem Storage

- `storage/objects/` — binary files (snapshots, recordings)
- `storage/meta/<uuid>.json` — `sidcar` metadata
- Served back via `GET /api/files/:fileId`

LiveKit SFU

`wss://live.talentstaq.ai`

- Room: `proctoring:{testId}:{attemptId}`
- Candidate publishes `cam/mic/screen` (`WebRTC`)
- Admin subscribes (`watch-only`) — media bypasses Express entirely

☐ Client (React) ☐ Backend (Node/Express) ☐ Python CV Service (FastAPI) ☐ Persistent storage ☐ LiveKit media pipeline ☐ External / optional

Key architectural fact: the frontend never talks to the Python CV service directly. All computer-vision calls are proxied through the Node backend (`pythonVisionService.ts`), keeping `PYTHON_CV_SERVICE_URL` an internal-only address. Two independent pipelines exist in parallel — (A) the periodic snapshot-analysis loop for violation detection, and (B) the LiveKit WebRTC pipeline for admins to watch live video — detailed on the following pages.

2 · Pipeline A — Snapshot Analysis & Violation Detection (real-time, synchronous)

- 1
- Frame capture (browser). `useProctoring.ts` grabs a webcam frame via `getUserMedia`, draws it to a `<canvas>`, and `proctorService.ts::captureFrame()` encodes it to base64 JPEG (max width 640px, quality 0.7).
- 2
- Client submits analysis. On an interval (`VITE_PROCTOR_ANALYSIS_INTERVAL_MS`, default 2500ms) the client calls `submitAnalysis()` → `POST /api/proctoring/session/:sessionId/analysis` with `{ timestamp, frameData, face, gaze, audio, objects, screenInfo, actualIntervalMs }`.
- 3
- Backend receives & rate-limits. `controllers/proctoring.ts::submitAnalysis` enforces a per-session minimum interval (`PROCTOR_SESSION_MIN_INTERVAL_MS`, default 3000ms) and a global concurrency cap (`PROCTOR_MAX_CONCURRENT_ANALYSIS`, default 140).
- 4
- Local heuristic pass. `proctorAIService.ts::analyzeProctoring()` checks client-submitted structured signals: face presence/count, gaze direction, phone-flag, fullscreen/tab-visibility. (Audio-level proctoring is currently disabled: `TEMP_DISABLE_AUDIO_PROCTORING = true`.)
- 5
- Frame proxied to Python CV service. If `frameData` is present, `pythonVisionService.ts::analyzeFrameWithPythonForSession()` POSTs `{ frame, sessionId, capturedAt }` to `{PYTHON_CV_SERVICE_URL}/analyze` (timeout ≥4500ms, retries=0 by design — next 2.5–4s cycle retries naturally; circuit breaker opens after 5–8 consecutive failures for 30–60s).
- 6
- CV inference (app.py, see box below). FastAPI dispatches CPU-bound work to a `ThreadPoolExecutor`; an `asyncio.Semaphore` caps in-flight analyses (returns `{overloaded:true}` immediately if saturated rather than queueing). Returns `violations / face / objects / stats` JSON.
- 7
- Merge, normalize, persist. Backend merges Python violations into the local list, normalizes event types (`EVENT_TYPE_ALIASES`), applies severity matrix, filters against an allow-list, attaches an evidence frame (current or short-TTL cache, 30s), then `storeViolation()` writes a `ProctorEvent` row and uploads the snapshot via `fileStorageService`.
- 8
- Real-time broadcast. Backend emits `violation-detected` / `candidate-status` over Socket.IO to room `proctor-{testId}`; admin dashboard updates live. `TestAttempt.violations` counter increments; a critical violation may auto-flag the attempt (`PROCTOR_AUTO_FLAG_ON_CRITICAL`).

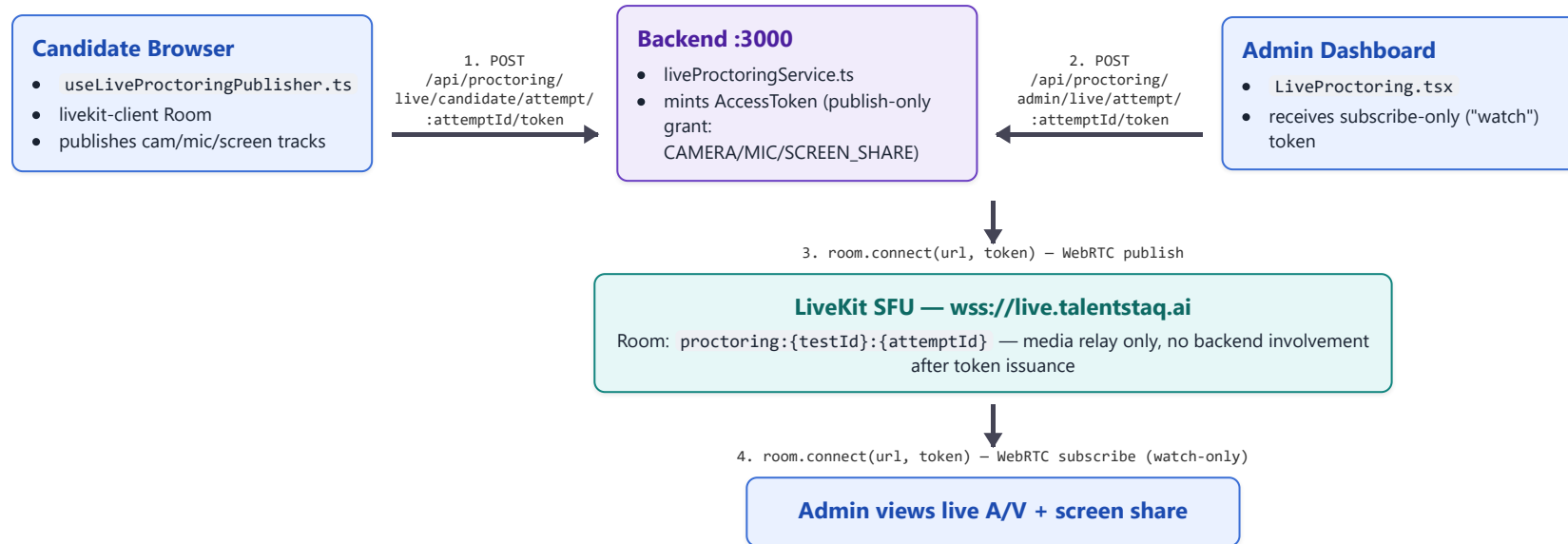
Inside app.py — CV detection techniques

Signal	Technique	Trigger threshold	Design detail	Behavior
Face presence / count	MediaPipe FaceDetection + FaceMesh consensus — present if <i>either</i> model detects a face; multi-face count only if <i>both</i> agree (avoids Haar-style double-count false positives)	no-face ≥5s, multi-face ≥3s	Per-session state	<code>_session_state</code> dict keyed by <code>sessionId</code> tracks duration timers so violations only fire after sustained conditions
Gaze / look-away	FaceMesh landmarks (eyes idx 33/263, nose idx 1) → horizontal offset ratio → center / left / right	ratio > 0.35, sustained ≥3s	Authoritative clock	Uses browser's <code>capturedAt</code> timestamp (not server-arrival time) to avoid inflating durations from network delay
Phone detection	YOLOv8n (ultralalytics), filtered to COCO class 67 "cell phone"	conf ≥0.35, area ≥0.00005 of frame	Gap reset	Timers reset if a gap exceeds the longest threshold — protects against false triggers from network drops / tab-hidden periods
Camera blocked	Plain OpenCV — mean grayscale brightness / std-dev	brightness < 18 (dark) or stddev < 8 (uniform/covered), ≥1.2s	Emit cooldown	<code>VIOLATION_COOLDOWN_SECONDS = 3</code> , <code>PHONE_EMIT_COOLDOWN_SECONDS = 3</code> , via <code>_should_emit</code>
			Enabled events	Default <code>CV_ENABLED_EVENTS = face_not_detected, multiple_faces, phone_detected</code> (looking_away / camera_blocked computed but off by default)

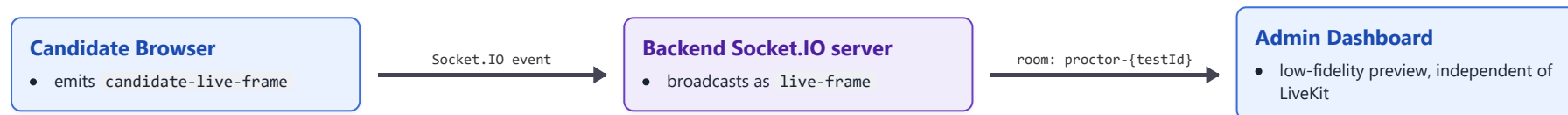
Note: `displayCount` / `bookCount` / `laptopCount` / `electronicCount` in the response are always 0 — placeholders, not wired to any model. Only phone detection is connected to YOLO. `ai_server.py` is a deprecated stub that raises `SystemExit` — `app.py` is the sole active CV service.

3 · Pipeline B — Live Video Watch (LiveKit WebRTC, parallel to Pipeline A)

A second, fully independent pipeline lets an admin watch a candidate's live camera/mic/screen feed. It does not touch violation detection at all — the backend's only role is minting short-lived LiveKit JWT tokens (control plane). Media itself flows directly between candidate and admin through the LiveKit SFU, bypassing Express entirely.



Lightweight fallback — raw frame preview over Socket.IO



Why two video paths exist: LiveKit gives full-quality, low-latency WebRTC for active "watch live" sessions; the Socket.IO raw-frame broadcast is a lightweight, always-on fallback/preview that doesn't require a LiveKit connection to be established. Neither path feeds back into Pipeline A's violation detection — CV analysis only happens on the periodic snapshot loop described on page 2.

4 · Data Model, Storage Layout & Configuration

POSTGRESQL — PRISMA MODELS (BACKEND/PRISMA/SCHEMA.PRISMA)

Model	Purpose / key fields
ProctorSession	attemptId (unique), camera/mic/screenShare enabled, browserInfo, deviceFingerprint, monitorCount, mobileVerified, startedAt/endedAt
ProctorEvent	eventType, severity, confidence(0-100), snapshotUrl, audioClipUrl, timestamp, reviewed/dismissed/reviewNotes
ProctorRecording	recordingType (webcam/screen/audio/combined), storageUrl/Key, duration, status (recording→uploading→ready)
ViolationClip	sub-clip of a recording tied to one violation (startOffset/endOffset)
FaceSnapshot	imageUrl, purpose (verification/periodic_check/violation_evidence), matchScore
ProctorTelemetry	raw JSON log per analysis/violation/engine-event cycle — audit/debug only

FILESYSTEM — BACKEND/STORAGE/ (FILE_STORAGE_MODE=FILESYSTEM)

```
storage/
├── objects/
│   ├── <uuid>.jpg / .webm           (flat, e.g. id_document)
│   └── Candidate Data/<candidateName>/
│       ├── recordings/<attemptId>-<type>-<ts>.webm
│       └── snapshots/<attemptId>-<type>-<ts>.jpg
└── meta/
    └── <uuid>.json   (sidecar metadata, 1 per object)
```

Example meta file — backend/storage/meta/45fb786a-....json :

```
{ "id": "45fb786a-...", "filename": "45fb786a-...jpg",
  "originalName": "...-selfie-1782989141551.jpg",
  "mimeType": "image/jpeg", "fileSize": 42019,
  "category": "id_document",
  "references": { "candidateId": "31b67f54-..." },
  "metadata": { "documentType": "selfie", "timestamp": 1782989141551 },
  "createdAt": "2026-07-02T10:45:41.555Z" }
```

Served back via GET /api/files/:fileId or GET /api/files/path/*. Alternative FILE_STORAGE_MODE=database stores binary bytes directly in Postgres FileStorage.data. No S3/cloud storage is wired up.

SERVICE PORTS (DEV)

Service	Port	Command
Backend (Express)	3000	npm run dev / node dist/index.js
Frontend (Vite)	5173	npm run dev / npm run preview
Python CV (FastAPI)	8010	uvicorn app:app --port 8010 [--workers 4]

KEY TUNABLES (ENV)

Var	Effect
PROCTOR_SESSION_MIN_INTERVAL_MS	Min gap between analyses per session (default 3000ms)
PROCTOR_MAX_CONCURRENT_ANALYSIS	Global in-flight cap (default 140)
PYTHON_CV_TIMEOUT_MS	Backend→Python call timeout (min enforced 4500ms)
PYTHON_CV_CIRCUIT_FAILURES / _OPEN_MS	Circuit breaker: opens after 5–8 fails, stays open 30–60s
CV_ENABLED_EVENTS	Which violation types app.py emits
PROCTOR_ENGINE_API_KEY	Auth for external AI engine webhook

BACKGROUND JOBS (SETINTERVAL IN INDEX.TS)

Job	Interval	Scope
snapshotTelemetry()	60s	Superadmin Observer telemetry — not CV
tickLiveTelemetry()	3s	Push live telemetry over Socket.IO
runAnomalyDetection()	10 min	Admin-account anomaly lock — security, not proctoring
runScheduledDeletions()	15 min	GDPR soft-delete cascade

No queue / async worker exists for proctoring. The .env file defines Redis-Streams and Gemma LLM-verifier variables (PROCTOR_CV_MODE="redis_streams" , PROCTOR_CV_REDIS_* , GEMMA_PROCTOR_*), but a full grep of backend/src found **zero** references to a Redis client or "gemma" — these are dead/unused config. Every CV analysis call is synchronous request/response (Node → Python /analyze per frame); there is no post-exam batch re-scan of recordings. Audio-level proctoring is currently disabled (TEMP_DISABLE_AUDIO_PROCTORING = true), and compareFaces() in proctorAIService.ts is a placeholder stub — real ID-selfie face matching is handled by a separate CompreFace service (COMPREFACE_* env vars, outside this proctoring pipeline).

5 · Recommended Architecture Improvements

The current design works and is well-commented, but several gaps surfaced during this review. Grouped below by effort/impact — quick wins first, then structural changes worth planning for as candidate volume grows.

QUICK WINS (LOW EFFORT, NO ARCHITECTURE CHANGE)

Area	Recommendation
Dead config	Remove or actually wire up <code>PROCTOR_CV_MODE="redis_streams"</code> , <code>PROCTOR_CV_REDIS_*</code> , <code>GEMMA_PROCTOR_*</code> — they currently exist in <code>.env</code> but nothing in <code>backend/src</code> reads them. Dead config misleads whoever debugs this next.
Stub cleanup	Delete <code>ai_server.py</code> (already disabled via <code>SystemExit</code>) and either implement or remove <code>compareFaces()</code> in <code>proctorAIService.ts</code> — it's a placeholder that could be mistaken for working code.
Response schema	<code>/analyze</code> always returns <code>displayCount/bookCount/laptopCount/electronicCount = 0</code> . Either wire these to the existing YOLO output (COCO already detects book/laptop/tv classes) or drop the fields so consumers don't assume they're live.
Retry policy	<code>PYTHON_CV_RETRY_COUNT=0</code> relies on the next capture cycle (2.5–4s later) to recover from a single dropped call. A single immediate retry with a short timeout would reduce missed detections during transient blips without materially raising load.
Audio proctoring	Currently globally disabled (<code>TEMP_DISABLE_AUDIO_PROCTORING=true</code>). Decide explicitly: re-enable with tuned thresholds, or remove the dead code path — a "temp" flag with no owner tends to stay off indefinitely.

RELIABILITY & SCALING (PYTHON CV SERVICE)

Area	Recommendation
Single point of failure	One <code>PYTHON_CV_SERVICE_URL</code> is configured; if it's one process/host, it's a single point of failure for all CV detection. Run 2+ instances behind a load balancer (or use <code>uvicorn</code> 's multi-worker mode already documented in <code>start.sh</code> , fronted by a proper LB, not just one host:port).
Actually use the queue	The unused Redis-Streams config suggests this was already planned — worth finishing: a queue between backend and CV service would absorb bursts, allow independent scaling of inference workers, and survive a CV-service restart without dropping in-flight frames (today a circuit-breaker window just means detection silently pauses for 30–60s).
GPU inference	YOLOv8n currently runs CPU-bound in a thread pool. At higher concurrent-candidate counts, a shared GPU worker (or batched inference across sessions) would cut per-frame latency and let one instance handle more sessions.

DATA, STORAGE & PRIVACY

Area	Recommendation
Move off local filesystem	Recordings/snapshots live on local disk (<code>backend/storage/</code>) with no redundancy. The README already flags this — prioritize DigitalOcean Spaces / S3 before scaling beyond a single backend host, since local storage also blocks horizontal-scaling the backend itself.
Encryption at rest	Candidate face snapshots and ID documents are stored as plain files. Given the biometric/PII sensitivity, encrypt at rest (or rely on the object store's server-side encryption once migrated off filesystem).
Retention policy	No visible TTL/retention job for proctoring recordings or snapshots (only admin soft-delete has a scheduled cascade). Define and automate a retention window for biometric evidence to limit compliance exposure.

DETECTION QUALITY & FEEDBACK LOOP

Area	Recommendation
No accuracy feedback loop	Admins already mark events <code>reviewed</code> / <code>dismissed</code> in <code>ProctorEvent</code> , but nothing feeds that signal back into threshold tuning. Periodically mining dismissed-vs-confirmed violations would help tune <code>PHONE_CONF</code> , gaze thresholds, etc. against real false-positive rates.
No post-exam re-scan	Full recordings are stored but never re-analyzed. A best-effort offline batch pass (using a heavier/more accurate model than the real-time YOLOv8n) over flagged or high-risk attempts would catch cases the real-time thresholds miss without adding real-time latency.
Model versioning	<code>dataset_db/</code> exists for YOLO fine-tuning but there's no visible eval harness. Track model version per <code>ProctorEvent</code> so future re-training can be validated against historical labeled events.

OBSERVABILITY & SECURITY

Area	Recommendation
Metrics dashboard	CV latency is already recorded (<code>telemetryRingBuffer.ts</code>) but there's no mention of it being exposed on a dashboard. Surface circuit-breaker state, per-session analysis latency, and overload-rejection counts (Prometheus/Grafana) so degraded CV performance is visible before candidates notice.
Webhook auth	The external-engine ingestion endpoint trusts a static <code>x-ai-key</code> header. Rotate it regularly and/or move to short-lived signed tokens if more third-party integrations are expected.

Overall theme: the real-time detection pipeline itself is solid and thoughtfully engineered (consensus face detection, capture-timestamp-based timers, circuit breakers). The biggest gaps are around **operational maturity at scale** — single-instance CV service, local-disk storage, and config that was clearly planned (Redis queue, Gemma verifier) but never finished or removed. Closing those would matter more than changing the detection logic itself.